



Методологии верстки

Методологии верстки

Поддерживать краткость CSS и HTML кода, делать его повторно используемым и обслуживаемым довольно тяжело.

Если пренебрегать какой-либо последовательностью в написании кода, то код может выйти из-под контроля, как в маленьких, так и средних и больших проектах, в которых задействовано более одного разработчика.

Рекомендуется использовать некоторую методологию верстки, которая позволит решить эти проблемы.

Методологии верстки

Существует множество методологий верстки и методологий стилизации:

- БЭМ,
- AMCSS,
- OOCSS,
- Atomic CSS,
- OPOR,
- MCSS,
- SMACSS,
- FUN,
- DoCSSa и пр.

Методология верстки БЭМ

БЭМ – это сокращение от **блок-элемент-модификатор**. Эта методология создана компанией Yandex в 2005 году.

Цель БЭМ - «разрабатывать сайты, которые необходимо быстро запустить и долго поддерживать. Методология помогает создавать расширяемые и повторно используемые компоненты интерфейса».

Основная концепция – легкая поддержка проектов со временем и повторное использование компонентов.

Главная стратегия БЭМ состоит в организации CSS-кода в повторно используемые модули с помощью умной системы именования.

Методология верстки БЭМ

Блок - это «функционально независимый компонент страницы, который может быть повторно использован. В HTML блоки представлены атрибутом class».

Нахождение блоков – решающий фактор применения методологии БЭМ.

Нужно ответить на вопрос: *Можно ли удалить кусок кода и использовать его где-то еще?*

Например, блоком будут **header** и **footer** сайта.

Блоки можно вкладывать друг в друга, использовать в любом месте на странице, поэтому в CSS-коде для блоков не должно быть никаких внешних отступов и правил позиционирования.

Методология верстки БЭМ

Имя блока должно описывать назначение блока, а не его внешний вид или состояние.

Имя должно отвечать на вопрос: *что это?* (например, **header**, меню и т.д.).

Вопрос не должен быть типа «как это выглядит» (например, **фиксированный header**, **маленькое меню** и т.д.).

Методология верстки БЭМ

Элемент – это «составная часть блока, которая не может использоваться в отрыве от него».

Принципы применения элементов:

- элементы «живут» только в блоках;
- элементы не могут принадлежать другим элементам, они могут лишь быть в блоках;
- можно создавать вложенные элементы;
- имена элементов описывают их назначение, а не внешний вид;
- при именовании элементов необходимо придерживаться стиля **block__element**.

Методология верстки БЭМ

Модификатор - это «сущность, определяющая внешний вид, состояние или поведение блока либо элемента».

Например, блок **footer** может быть зафиксирован в верхней части страницы, блок кнопки может быть отключен и т.д.

Стиль именования модификаторов в БЭМ: **block__element_modifier**.

Методология верстки БЭМ

Также БЭМ предлагает принципы организации структуры файлов, набор инструментов и живое сообщество с поддержкой.

Плюсы БЭМ

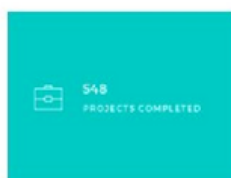
- новые разработчики могут быстро понять связь между компонентами в разметке и CSS;
- методология способствует повышению производительности в команде, преимущества особенно заметны в крупных проектах;
- система именования снижает риски коллизий с классами;
- CSS несильно привязан к разметке в определенном месте на странице;
- CSS становится повторно используемым.

Методология БЭМ, пример

Рассмотрим следующий фрагмент верстки:



На рисунке четыре одинаковых блока с разным содержимым выводятся в одну строку. Рассмотрим структуру каждого блока:



В этом блоке есть:

- Фотография
- Название
- Описание

Методология БЭМ, блоки



548
PROJECTS COMPLETED

Блоки

Отнесем этот блок к классу **prof-skills**:

```
<div class="prof-skills"></div>
```

В смысле методологии БЭМ – это **блок**, так как по определению это «функционально независимый компонент страницы, который может быть повторно использован».

Если имя блока состоит из нескольких слов, эти слова записываются через «-».

Методология БЭМ, элементы



548
PROJECTS COMPLETED

Элементы

Определим **элементы** в блоке (по определению «элемент - ставная часть блока, которая не может использоваться в отрыве от него»). В этом блоке таких элементов 3:

- фотография;
- название;
- описание.

Каждый из них должен относиться к определенному классу, имя которого образуется так:

имя - блока__имя - элемента



```
<!-- блок -->
<div class="prof-skills">
  <!-- элементы -->
  <div class="prof-skills__img"></div>
  <div class="prof-skills__title"></div>
  <div class="prof-skills__desc"></div>
</div>
```

Методология БЭМ, элементы



На элементы накладываются следующие **ограничения**:

- элемент одного блока не может использоваться в другом

```
<div class="prof-skills">
  <div class="prof-skills__img"></div>
```

```
<!-- Нельзя, так как этот элемент принадлежит блоку
about -->
```

```
  <div class="about__title"></div>

  <div class="prof-skills__desc"></div>
</div>
```

- Нельзя использовать элемент блока вне самого блока. Например такая запись запрещена, так как элемент блока "prof-skills" находится вне своего блока.

```
<div class="prof-skills">
  <div class="prof-skills__img"></div>
  <div class="prof-skills__title"></div>
  <div class="prof-skills__desc"></div>
</div>
```

```
<!-- такая запись запрещена -->
<div class="prof-skills__title"></div>
```

- разрешено создавать вложенные элементы, но нельзя наследовать их от родительских элементов:

```
<!-- запрещено -->
<div class="prof-skills">
  <div class="prof-skills__img">
    <div class="prof-skills__img__user-img"></div>
  </div>
  <div class="prof-skills__title"></div>
  <div class="prof-skills__desc"></div>
</div>
```


- разрешено создавать вложенные элементы, но нельзя наследовать их от родительских элементов:

```
<!-- разрешено -->
<div class="prof-skills">
  <div class="prof-skills__img">
    <div class="prof-skills__user-img"></div>
  </div>
  <div class="prof-skills__title"></div>
  <div class="prof-skills__desc"></div>
</div>
```

Методология БЭМ, модификаторы

Модификаторы

Определим для нашего блока **модификаторы**. Простыми словами модификаторы используются, чтобы указать какую-то активность. Например, нажатие кнопки, наведение курсора мыши и пр.

Модификаторы оформляются так:

имя - блока__имя - элемента_имя - модификатора .

Методология БЭМ, модификаторы

HOME

ABOUT

WORK

PROCESS

SERVICES

TESTIMONIALS

CONTACT

```
<div class="header">
  <a href="#" class="header__item
header__item_active">Home</a>
  <a href="#" class="header__item">About us</a>
  <a href="#" class="header__item">Contact</a>
</div>
```

Элементу **a**, класса **header__item**, в котором содержится текст «Home», добавлен модификатор «**active**». Этот модификатор относится к классу **header__item_active** и позволяет выделить «текущую» страницу в меню.

Методология БЭМ, миксы

Миксы

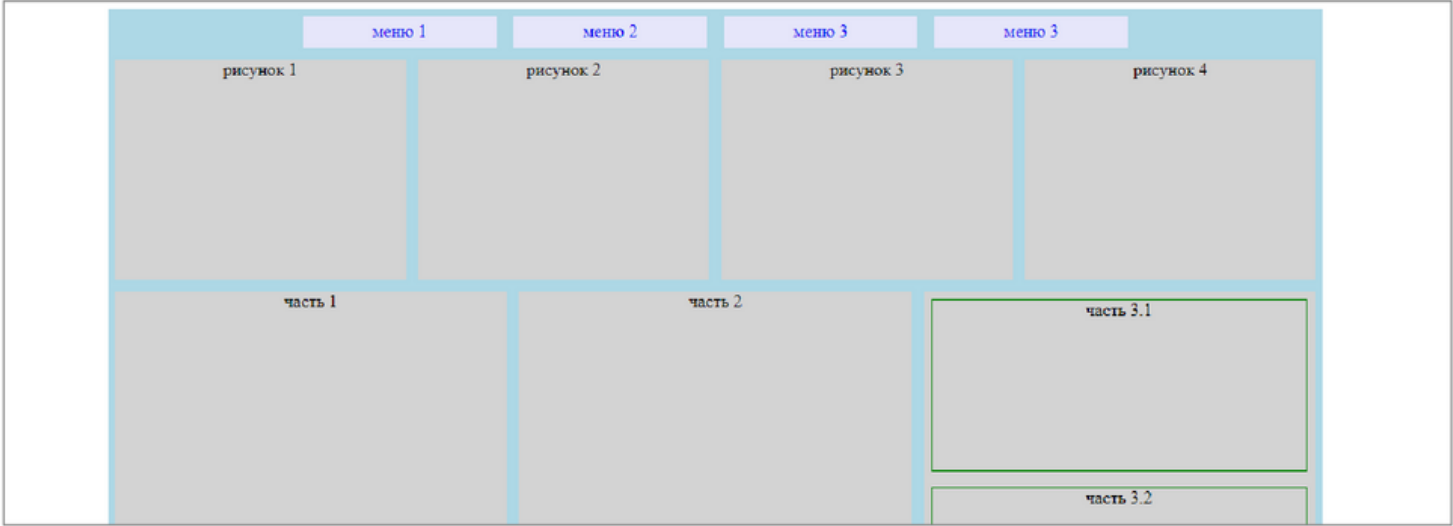
Миксы нужны для того, чтобы можно было заимствовать свойства у блока, но при этом оставить блок независимым.

```
<div class="prof-skills">
  <div class="prof-skills__img"></div>
  <div class="prof-skills__title title"></div>
  <div class="prof-skills__desc"></div>
</div>
```

В этом коде в элемент **prof-skills__title** добавлен класс блока **title**. Элемент **prof-skills__title** заимствует все стили у **title**, но при этом можно менять их. Таким образом можно добавить стили в **prof-skills__title**, но при этом не портить стили **title**.

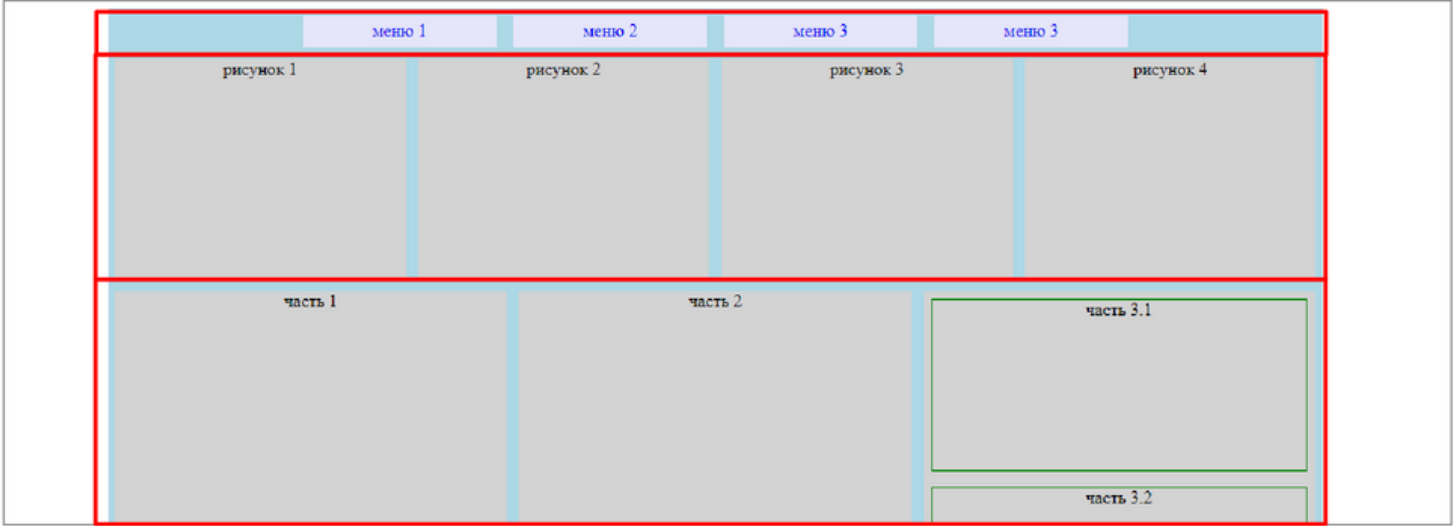
БЭМ, пример верстки

Пример. Сверстать страницу с использованием методологии БЭМ



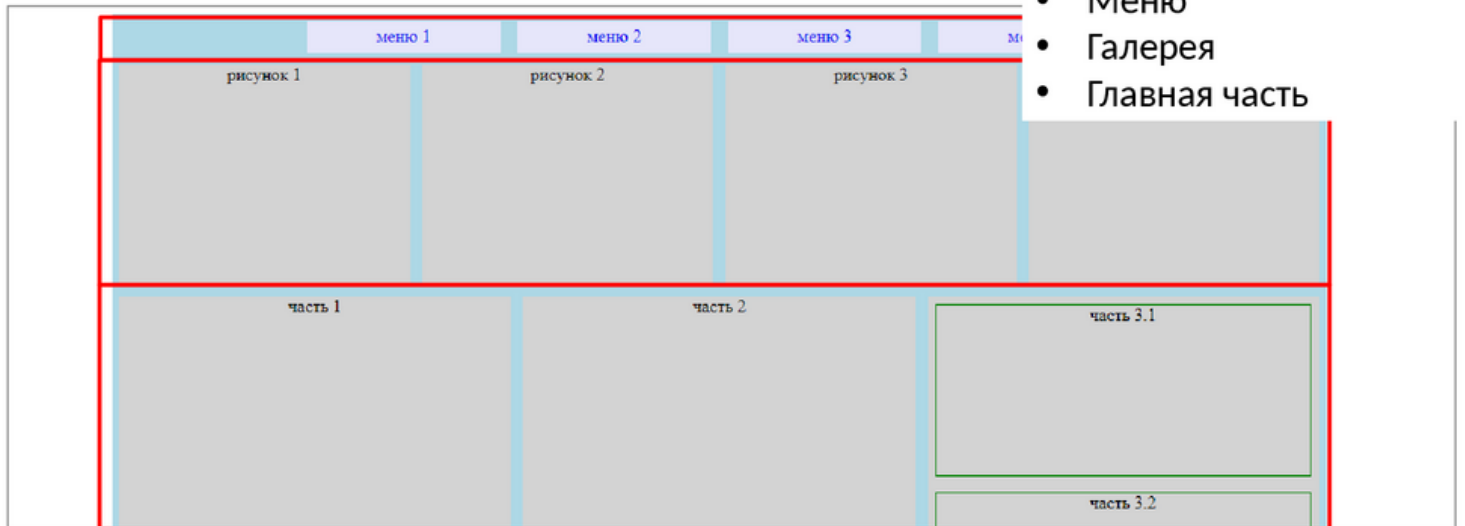
БЭМ, пример верстки

Шаг 1. Разбиваем страницу на блоки



БЭМ, пример верстки

Шаг 1. Разбиваем страницу на блоки



БЭМ, пример верстки

Шаг 1. Реализуем соответствующий HTML-код

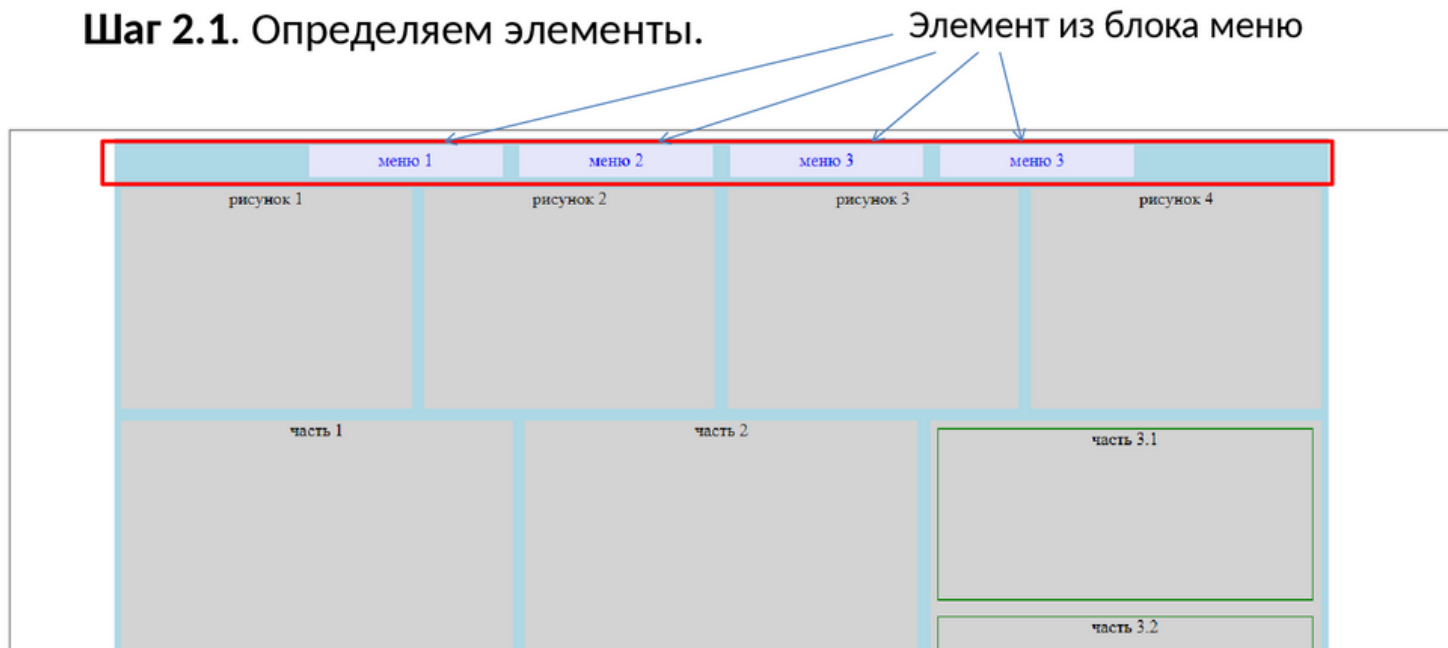
```
<body>
  <div class="menu">
  </div>

  <div class="gallery">
  </div>

  <div class="main">
  </div>
</body>
```


БЭМ, пример верстки

Шаг 2.1. Определяем элементы.



БЭМ, пример верстки

Шаг 2.1. Реализуем соответствующий HTML-код

```
<body>
  <div class="menu">
    <a href="#" class="menu__link">меню 1</a>
    <a href="#" class="menu__link">меню 2</a>
    <a href="#" class="menu__link">меню 3</a>
    <a href="#" class="menu__link">меню 4</a>
  </div>

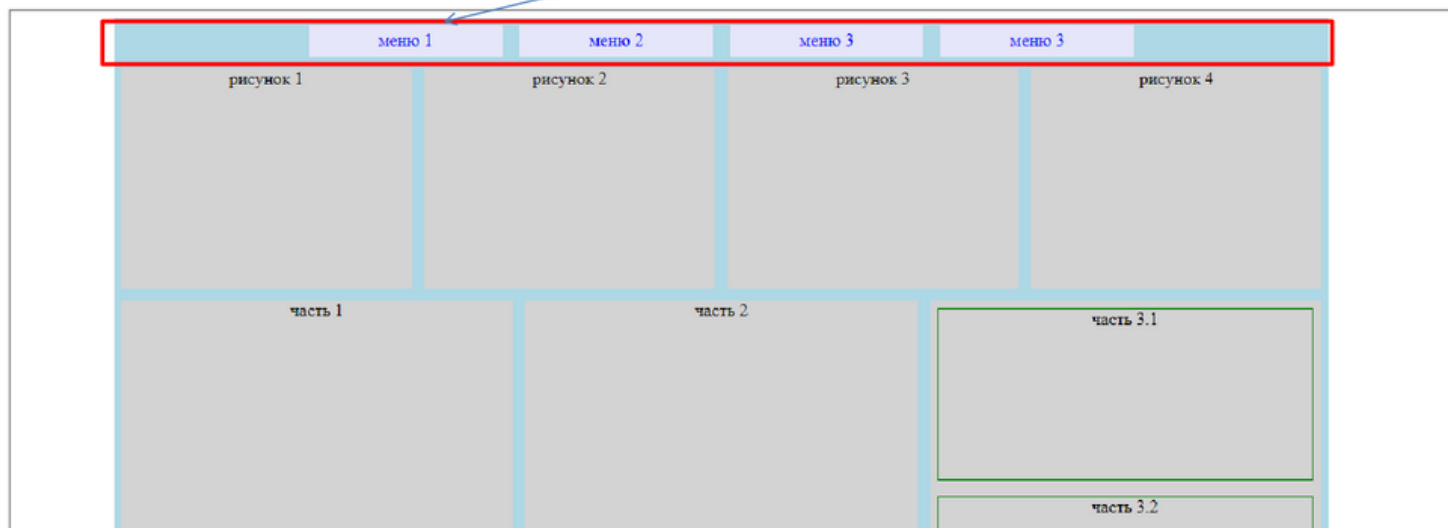
  <div class="gallery">
  </div>

  <div class="main">
  </div>
</body>
```

БЭМ, пример верстки

Шаг 2.1. Определяем модификаторы.

Элемент для текущей страницы



БЭМ, пример верстки

Шаг 2.1. Реализуем соответствующий HTML-код

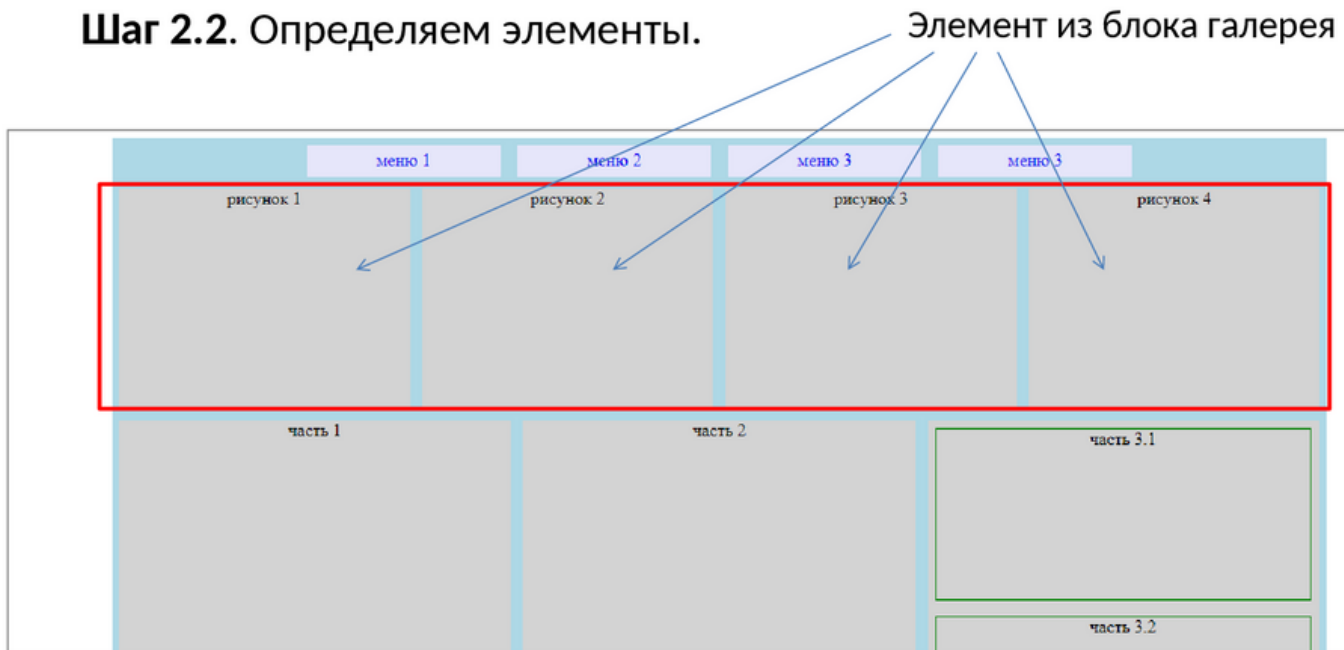
```
<body>
  <div class="menu">
    <a href="#" class="menu__link menu__link_active">
      меню 1
    </a>
    <a href="#" class="menu__link">меню 2</a>
    <a href="#" class="menu__link">меню 3</a>
    <a href="#" class="menu__link">меню 4</a>
  </div>

  <div class="gallery">
  </div>

  <div class="main">
  </div>
</body>
```

Методология верстки БЭМ

Шаг 2.2. Определяем элементы.



Методология верстки БЭМ

Шаг 2.2. Реализуем соответствующий HTML-код

<body>

...

```
<div class="gallery">  
    
    
    
    
</div>
```

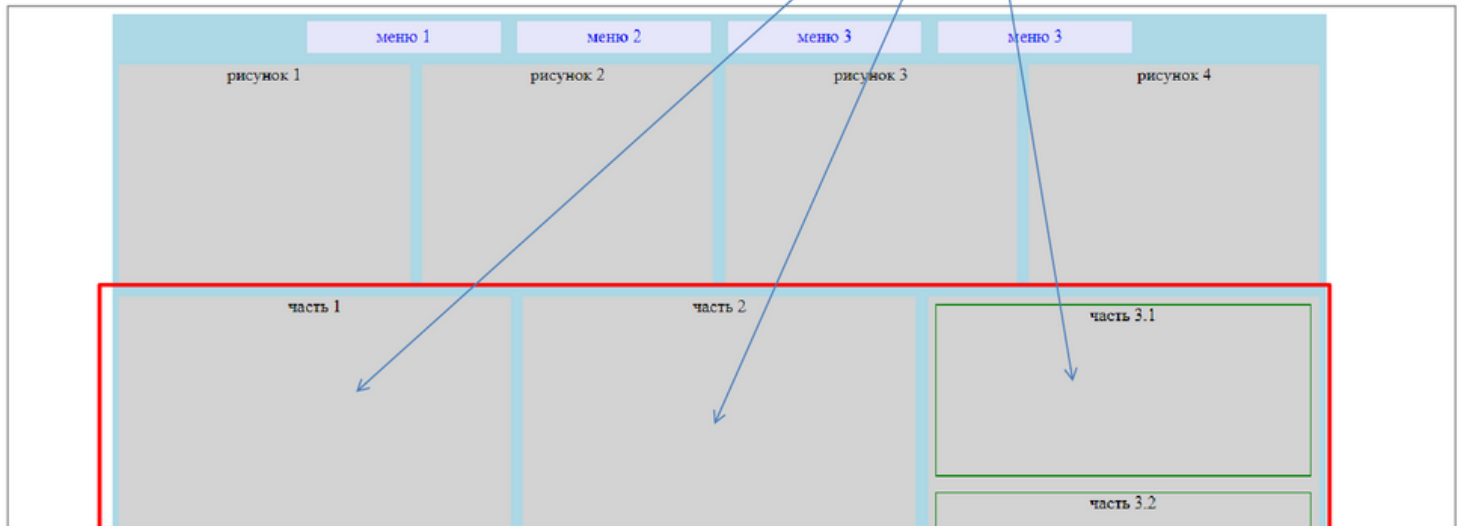
...

</body>

БЭМ, пример верстки

Шаг 2.3. Определяем элементы.

Элемент из блока главная часть



БЭМ, пример верстки

Шаг 2.3. Реализуем соответствующий HTML-код

```
<body>
  <div class="menu">
    ...
  </div>

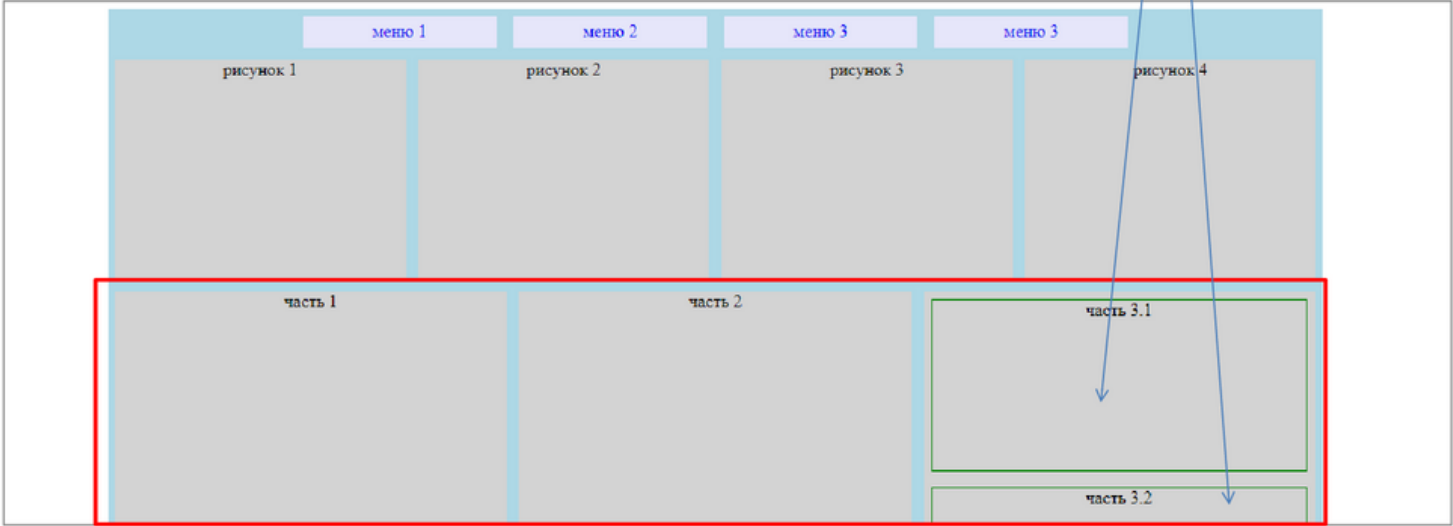
  <div class="gallery">
    ...
  </div>

  <div class="main">
    <div class="main__article">часть 1</div>
    <div class="main__article">часть 2</div>
    <div class="main__article">часть 3</div>
  </div>
</body>
```

БЭМ, пример верстки

Шаг 2.4. Определяем вложенные элементы.

Вложенный элемент



БЭМ, пример верстки

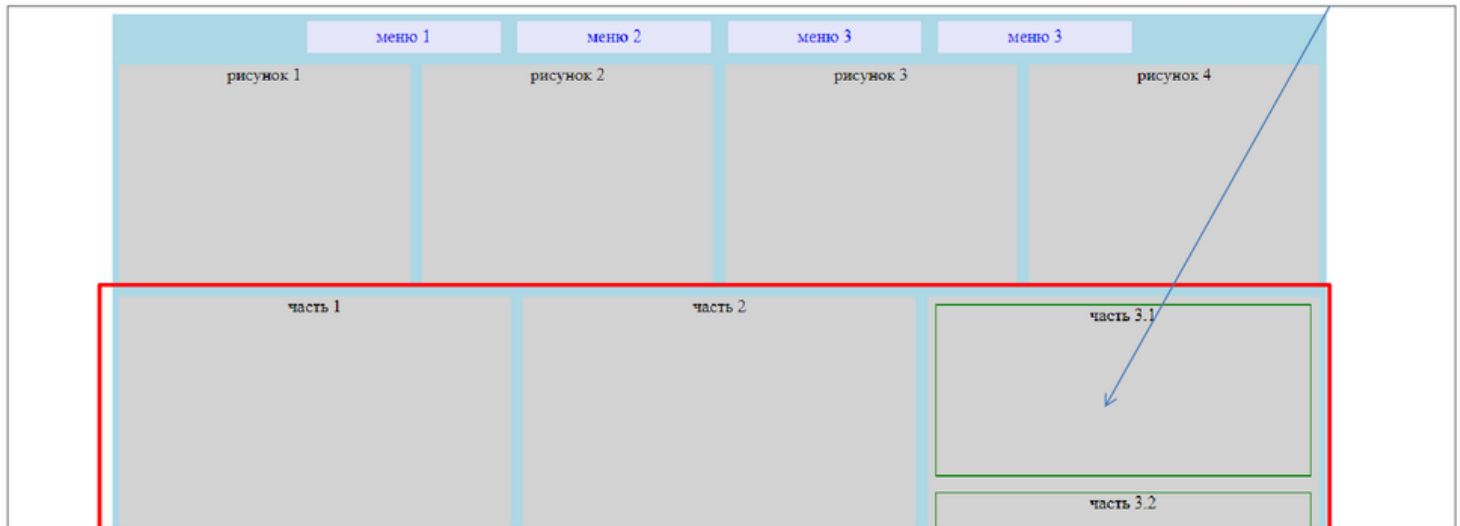
Шаг 2.4. Реализуем соответствующий HTML-код

```
<body>
...
  <div class="main">
    <div class="main__article">часть 1</div>
    <div class="main__article">часть 2</div>
    <div class="main__article">
      <div class="main__article-topic">часть 3.1</div>
      <div class="main__article-topic">часть 3.2</div>
      <div class="main__article-topic">часть 3.3</div>
    </div>
  </div>
</body>
```


БЭМ, пример верстки

Шаг 2.4. Определяем модификаторы.

Предположим, первый как-то выделен, например, цветом

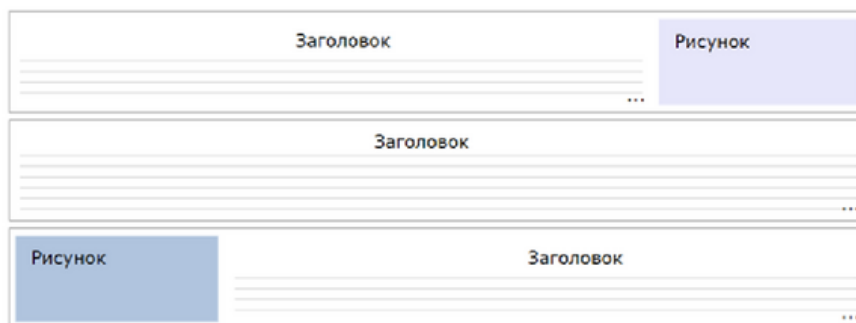


Методология верстки БЭМ

Шаг 2.4. Реализуем соответствующий HTML-код

```
<body>
...
  <div class="main">
    <div class="main__article">часть 1</div>
    <div class="main__article">часть 2</div>
    <div class="main__article">
      <div class="main__article-topic
        main__article-topic_first">
        часть 3.1
      </div>
      <div class="main__article-topic">часть 3.2</div>
      <div class="main__article-topic">часть 3.3</div>
    </div>
  </div>
</body>
```

Задание 1. Сверстать фрагмент страницы с использованием методологии БЭМ.



Методология верстки SMACSS

SMACSS (масштабируемая и модульная архитектура CSS) – методология веб-разработки для организации и написания CSS-кода.

Создатель Jonathan Snook описывает ее так:

«SMACSS – это способ изучить ваш процесс проектирования, а также способ приспособить эти жесткие рамки под гибкий мыслительный процесс. Это попытка документирования последовательного подхода к разработке сайта с использованием CSS».

В ядре лежит способ разделения CSS-правил на категории. Категории порождают шаблоны, т.е. повторяющиеся элементы в дизайне, вокруг которых можно выработать правила написания обслуживаемого и повторно используемого CSS.

Методология верстки SMACSS

Базовые категории SMACSS:

- **Base** – в эту категорию входят правила, которые определяют внешний вид элементов по умолчанию. Одиночные селекторы элементов, селекторы атрибутов, селекторы псевдоклассов, смежные селекторы и т.д.

```
body {  
    padding: 0;  
    margin: 0;  
}  
a {  
    text-decoration: none;  
}  
a:hover {  
    text-decoration: underline;  
}
```

Методология верстки SMACSS

Базовые категории SMACSS:

- **Layout** – категория для стилей, с помощью которых страница разделяется на секции.

```
.l-container-row {  
    margin: 0 auto;  
    display: flex;  
    min-width: 230px;  
}  
  
.l-container-column {  
    margin: 0 auto;  
    display: flex;  
    flex-direction: column;  
    min-width: 230px;  
}
```

Методология верстки SMACSS

Базовые категории SMACSS:

- **Module** – модули – это повторно используемые части дизайна, как в лего. Например, меню, диалоговые окна, поисковой блок и т.д. Допускаются дочерние элементы с префиксом из класса родительского элемента через дефис.

```
.topic {  
  display: grid;  
  grid-template-rows: 1fr 2fr;  
  grid-template-columns: 1fr 2fr;  
  grid-template-areas:  
    "image title"  
    "image text"  
}
```

```
.topic > .topic-image {  
  grid-area: image;  
}  
.topic > .topic-title {  
  grid-area: title;  
}  
.topic > .topic-text {  
  grid-area: text;  
}
```

Методология верстки SMACSS

Базовые категории SMACSS:

- **State** – к этой категории относятся стили внешнего вида макета или модулей в определенном состоянии (например, видимый, скрытый, раскрытый или закрытый) или в определенном виде (например, домашняя страница или внутренняя страница).

```
.is-selected {  
  background-color: lightgrey;  
}
```

Методология верстки SMACSS

Базовые категории SMACSS:

- **Theme rules** – категория, в которую входят стили, отвечающие за внешний вид макетов и модулей.

```
.fill-stroke {  
  border: 1px solid red;  
  background: rgba(255, 0, 0, 0.5);  
}
```

Методология верстки SMACSS

Система именования SMACSS

К правилам **Layout**, **State** и **Module** добавляется префикс с осмысленным именем или сокращением:

- для правил **Layout** можно использовать префикс **layout-**, **l-**;
- для правил **State** необходимо добавлять префикс состояния **is-**, например, **is-hidden**, **is-visible** и т.д.
- для **Module** используются сами имена компонентов, например, **.menu**, **.dialog** и т.д.
- связанные элементы внутри модуля должны использовать имя модуля в качестве префикса. Например, чтобы выбрать пункт меню в модуле **menu**, используются что-то типа **.menu-link** или **.menu-item**, а не **.menu li a**.

Методология верстки SMACSS

Система именования SMACSS

Связанные элементы внутри модуля и вариации модуля должны использовать базовое имя модуля в качестве префикса.

Например, чтобы выбрать пункт меню в модуле **menu**, используются что-то типа **.menu-link** или **.menu-item**, а не **.menu li a**.

Методология верстки SMACSS

В отличие от БЭМ, SMACSS не предписывает слишком строгое соглашение об именованиях. Jonathan Snook дал четко понять: «... не думайте, что нужно жестко придерживаться этих правил. У вас есть соглашение, задокументируйте и придерживайтесь его».

Преимущества SMACSS

- подход предлагает правильные рекомендации для модульного и поддерживаемого CSS-кода, избегая при этом излишних предписаний;
- SMACSS быстро выучить (и научить);
- система именования SMACSS менее подробна и в чем-то проще БЭМ;
- система достаточно гибкая, чтобы работать с крупными и маленькими проектами.

Методология верстки ECSS

Enduring CSS (выносливый CSS) или eCSS это: «Руководство к написанию стилей для больших, быстро меняющихся, долгоживущих веб-проектов». Автор Ben Frain.

Центральная концепция eCSS – изоляция. Изоляция значит, что все компоненты представляют собой изолированную единицу кода без зависимостей, контекста.

Эту единицу можно использовать повторно и удалять без риска утечки стилей.

Методология верстки ECSS

В основном это достигается за счет:

- инкапсуляции всего кода, не только CSS, а всех технологий, необходимых для постройки всех компонентов в своих общих папках;
- создания совершенно нового компонента каждый раз, когда нужен компонент, похожий на уже существующий, но в котором будут некоторые изменения, даже если изменения незначительны;
- использования строгой системы именования.

Методология верстки ECSS

Пункт о создании нового компонента означает, что повторение свойств и значений для eCSS не проблема.

В этом плане eCSS радикально отходит от таких методологий, как БЭМ и SMACSS, которые расширяют и абстрагируются от существующих компонентов, тем самым избегая или пытаясь как можно сильнее избежать повторения кода.

Значит ли это, что eCSS утяжеляет файлы стилей? После нескольких тестов по сжатию файлов Ben Frain пришел к выводу, что из-за того, что gzip невероятно эффективно сжимает повторяющиеся строки, разница в весе файла между eCSS и другими методологиями крошечная.

Методология верстки ECSS

Преимущества eCSS

- изоляция всех визуальных шаблонов облегчает обслуживание CSS-кода;
- несмотря на повторяющиеся свойства и значения, размер файла в долгосрочной перспективе почти не увеличится. Это происходит потому, что модули являются автономными, изолированными единицами, которые можно быстро удалить без страха поломать дизайн, если они больше не нужны;
- все языковые/технические файлы, необходимые для создания модуля, расположены в одной папке, что сильно упрощает редактирование и физическое удаление.

Выводы

- использование методологий верстки необходимо;
- можно выбрать уже существующую;
- можно совместить методологии, например, БЭМ и SMACSS;
- можно выработать свою методологию на основе определенных проблем, которые вам необходимо решить, задокументировать ее и использовать.

Спасибо за внимание!